

HTBLuVA Graz –Gösting
Ibererstraße 15-21
8051 Graz



Abteilung für Elektronik und Technische Informatik

Black Mirror

Ausgeführt im Schuljahr 2025/26 von:

Robert Milanovici
Manuel Greinix

5BHEL
5BHEL

Betreut von:

DI Franz Patz
DI Rudolf Schamberger

Vorgelegt am tt. Monat jjjj

Inhaltsverzeichnis

	Seite
1 Einleitung	6
1.1 Kurzfassung	6
1.2 Abstract	6
2 Lastenheft	7
2.1 Projektziel	7
2.2 Nutzen des Produkts	7
2.3 Randbedingungen	8
2.4 Chancen und Risiken	8
2.5 Aufwandsabschätzung	8
3 Projektplanung	9
3.1 Aufgabenteilung	9
4 Grundlagen	10
4.1 Bildverarbeitung	10
4.1.1 Grauwertbild	10
4.1.2 Klassische Bildverarbeitung	10
4.1.3 Segmentierung	11
4.1.4 Zeitliche Glättung	11
4.2 Künstliche Intelligenz in der Bildanalyse	11
4.2.1 Mediapipe Selfie Segmentation	11
4.2.2 Mediapipe Hands	12
4.3 LED-Display-Technik	12
4.4 Git, Versionsverwaltung und Entwicklungsumgebung	12
5 Analyse und Anforderungen	13
5.1 Ausgangssituation und Ideenfindung Black Mirror	13
5.2 Funktionale Anforderungen	13
5.3 Use Cases und Szenarien	13

5.4 Systemgrenzen und Annahmen	13
6 Evaluierte Lösungsansätze	14
6.1 Personenerfassung	14
6.1.1 Bewegungs -und Kantenerkennung	14
6.1.2 BodyPix KI-Segmentierung	14
6.1.3 ASCII-Mirror	14
6.2 Led-Signal	15
6.2.1 Mehr SPIs	15
6.2.2 Schmitt Trigger Verstärkerschaltung	15
6.3 Finaler Algorithmus	17
6.4 Case.....	17
7 Verwendete Technologien	18
7.1 Programmiersprache und Bibliotheken (Python, OpenCV, NumPy)	18
7.2 MediaPipe Selfie Segmentation und Hands	18
7.3 Bildverarbeitungs- und Filterverfahren	19
7.4 LED-Ansteuerung.....	19
7.5 Entwicklungswerkzeuge.....	20
8 Systemkonzept und Architektur.....	21
8.1 Gesamtüberblick des Systems.....	21
8.2 Hardwarearchitektur.....	21
8.2.1 Kameramodul.....	21
8.2.2 Raspberry Pi	22
8.2.3 MAX7219	22
8.2.4 Case.....	23
8.2.5 Weiter Komponente	23
8.3 Softwarearchitektur	24
8.3.1 Personenerfassung	24
8.3.2 LED-Ansteuerung.....	24
9 Personenerfassung Implementierung	25
9.1 Importe und Kameraauswahl	25

9.2 Kameraauswahl	25
9.3 Kamerainitialisierung & Auflösung.....	26
9.4 Selfie Segmentation	26
9.5 Hand-Tracking	27
9.6 Hintergrundmodell & Kalibrierungsphase.....	27
9.7 Helligkeitsanpassung	28
9.8 Personensegmentierung	28
9.9 Handerkennung & Umrechnung der Landmarken.....	29
9.10 Handflächenaufbau	29
9.11 Finger, Verbindungen und Gelenke	30
9.12 Morphologische Bearbeitung der Maske	30
9.13 Zeitliche Glättung	31
9.14 Maskenfusion & Bereinigung	31
9.15 Konturen & finale Glättung	32
9.16 Ausgabe	33
10 Led-Ansteuerung Implementierung	34
10.1 Importieren der Bibliotheken	34
10.2 Plattformerkenung.....	34
10.3 Initialisierung der LED-Matrix (nur auf Raspberry Pi).....	34
10.4 Hauptfunktion zur Bilddarstellung.....	35
10.5 Aufteilen des Bildes in Blöcke	35
10.6 Drehen einzelner Blöcke	35
10.7 Umwandlung in Graustufen & Convert to PIL-Image	36
10.8 Finale Anzeige	36
10.9 Anmerkung.....	36
11 Elektronische Umsetzung der Led-Matrix	37
11.1 Verbindungsplatine	37
11.1.1 Layout	37
11.1.2 Platine	37
11.2 Verstärkerplatine	38

11.2.1	OPA354AIDBVT.....	38
11.2.2	Schaltung auf Multisim	39
11.2.3	Simulation der Verstärkerschaltung	40
11.2.4	Layout	41
11.2.5	Platine	41
11.3	Levelshifterplatine	42
11.3.1	SN74AHCT125N.....	42
11.3.2	Schaltung auf Multisim	42
11.3.3	Simulation	43
11.3.4	Layout	44
11.3.5	Platine	44
12	Mechanischer Aufbau und Design	45
12.1	Gehäusekonzept und Spiegelaufbau	45
12.2	Montage von Kamera und LED-Matrix	45
12.3	Kabelmanagement und Sicherheit	45
12.4	Designüberlegungen	45
13	Tests und Evaluierung	46
13.1	Teststrategie und Testumgebung.....	46
13.1.1	Personenerfassung	46
13.1.2	Led-Ansteuerung.....	46
13.1.3	Signalübertragung.....	46
13.2	Funktionstests	47
13.2.1	Personenerfassung	47
13.2.2	Led-Ansteuerung.....	47
13.3	Performance-Messungen.....	47
13.3.1	Personenerfassung	47
13.3.2	Led-Ansteuerung.....	47
13.3.3	Signalübertragung.....	47
13.4	Auswertung der Testergebnisse	48
14	Wirtschaftliche Betrachtung	49
14.1	Kostenanalyse.....	49

15 Zusammenfassung und Ausblick	51
15.1 Zusammenfassung der Ergebnisse.....	51
15.2 Erfüllung der Zielsetzung	51
15.3 Mögliche Erweiterungen.....	51
15.4 Persönliche Reflexion	51
16 Literaturverzeichnis.....	52
17 Abbildungsverzeichnis	53
18 Eidesstattliche Erklärung.....	55

1 Einleitung

1.1 Kurzfassung

Unsere Diplomarbeit ist die Entwicklung der Hardware und Software für unseren Black Mirror. Der Black Mirror ist eine LED-Fläche mit einem Kameramodul, welche die Person in Form eines abstrahierten Spiegelbildes darstellt. Durch das gezielte Ein- und Ausschalten der LEDs entsteht ein visueller Effekt, der dem Prinzip des ASCII-Mirrors ähnelt. Anstatt Zeichen zu verwenden, nutzen wir LED-Matrizen, wodurch eine physische und hardwarebasierte Umsetzung entsteht. Wir haben uns für die Entwicklung des Black Mirrors entschieden, da wir dafür sowohl Software- als auch Hardwareentwicklung umsetzen mussten. Eines unserer wichtigsten Ziele ist es, die LEDs mit so wenig Verzögerung wie möglich anzusteuern, dabei aber dennoch effizient zu arbeiten. Das Herz des Black Mirrors, welches alle Bauteile miteinander verbindet, ist der Raspberry Pi 5. Für das Kameramodul haben wir uns für das Camera Module 3 Wide NoIR entschieden, welches von Raspberry Pi entwickelt wurde.

1.2 Abstract

Our diploma thesis is the development of the hardware and software for our Black Mirror. The Black Mirror is an LED surface with a camera module that represents the person in the form of an abstract mirror image. By selectively turning the LEDs on and off, a visual effect is created that resembles the principle of the ASCII mirror. Instead of using characters, we use LED matrices, resulting in a physical and hardware-based implementation. We decided to develop the Black Mirror because it required us to carry out both software and hardware development. One of our most important goals is to control the LEDs with as little delay as possible while still maintaining efficiency. The heart of the Black Mirror, which connects all components, is the Raspberry Pi 5. For the camera module, we chose the Camera Module 3 Wide NoIR, which was developed by Raspberry Pi.

2 Lastenheft

2.1 Projektziel

Ziel des Projekts ist die Entwicklung eines Systems namens Black Mirror, das mithilfe einer Kamera Personen oder Objekte erkennt und deren abstrahierte Darstellung in Echtzeit auf einer LED-Matrix visualisiert.

Das System gilt als erfolgreich umgesetzt, wenn eine stabile Erkennung sowie eine flüssige und verzögerungsfreie Darstellung auf der LED-Matrix erreicht wird.

Durch die Kombination aus Computer Vision, Elektronik und interaktiver Visualisierung entsteht ein technisch und gestalterisch anspruchsvolles Projekt. Die Umsetzung erfolgt mit gängigen Komponenten wie Raspberry Pi, Kamera und LED-Matrix innerhalb eines Entwicklungszeitraums von etwa 250–300 Stunden.

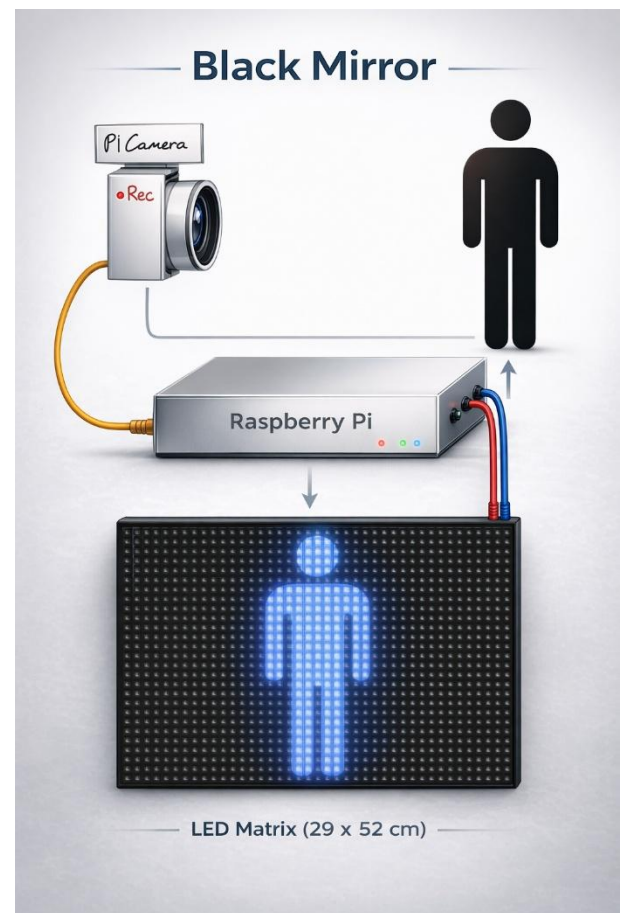


Abbildung 1: Abstrahierte Darstellung

2.2 Nutzen des Produkts

Der Black Mirror verbindet moderne Bilderkennung mit einer künstlerischen Echtzeit-Darstellung. Das System kann als interaktives Display in Ausstellungen, öffentlichen Räumen oder im Smart-Home-Bereich eingesetzt werden.

Der Mehrwert liegt in der Kombination aus Echtzeit-Erkennung und großflächiger visueller Darstellung, wodurch ein hoher Wiedererkennungswert entsteht.

2.3 Randbedingungen

Die Umsetzung erfolgt mit festgelegten Hardwarekomponenten wie Kamera, LED-Matrix und Netzteil innerhalb eines begrenzten Budgets. Die Systemgröße sowie die verfügbaren Ressourcen stellen zusätzliche Rahmenbedingungen dar.

2.4 Chancen und Risiken

Das Projekt besitzt Potenzial im Bereich interaktiver Displays und künstlerischer Installationen. Risiken bestehen insbesondere in der technischen Komplexität, der Marktakzeptanz sowie möglichen Hardware-Limitierungen.

2.5 Aufwandsabschätzung

Die Entwicklung des Prototyps erfolgt durch ein zweiköpfiges Team mit Kenntnissen in Programmierung, Bildverarbeitung und Hardwareintegration.

Die geschätzten Materialkosten für den Prototypen liegen bei etwa 350 €, während der gesamte Entwicklungsaufwand rund 250–300 Stunden beträgt.

3 Projektplanung

3.1 Aufgabenteilung

Die Umsetzung des Projekts wurde auf zwei Personen aufgeteilt.

Robert war hauptsächlich für die Entwicklung der Personenerfassung verantwortlich. Zur Personenerkennung gehört die Optimierung der Bildverarbeitungsverfahren sowie die Integration KI-basierter Segmentierungen. Dazu war seine Aufgabe die Auswahl der benötigten Bauteile für den Black Mirror und die Simulation sowie der Integration der Hardwaremodule, besonders bei den Led-Matrizen und die dazugehörige Elektronik.

Manuel war für die Live-Übertragung des Kamerabildes auf die Led-Matrix sowie für das Design und die Erstellung des Gehäuses („Case“) und verschiedener Platinen verantwortlich. Darüber hinaus übernahm er die Dokumentation. Die Bildübertragung umfasste die Entwicklung eines funktionierenden Python-Codes sowie dessen optimale Abstimmung auf die Led-Matrix. Der Designvorgang der Platinen wurde hauptsächlich in Fusion360 fabriziert. Die Dokumentation erfolgte sowohl auf den von der Schule vorgegebenen Plattformen als auch in einem vom Team erstellten GitHub-Projekt.

4 Grundlagen

4.1 Bildverarbeitung

4.1.1 Grauwertbild

Für dieses Projekt wird das Farbbild zunächst in Graustufen umgewandelt. Dabei werden die Pixel so umgewandelt, dass sie nur noch einen Helligkeitswert besitzen. Dies wird oft verwendet in Bewegungs- und Hintergrunds Analyse. [1]

4.1.2 Klassische Bildverarbeitung

Zu den klassischen Bildverarbeitungen gehören Filter, Thresholding und morphologische Filter.

- Thresholding wird insbesondere verwendet, damit im Bild in Vor- und Hintergrund getrennt wird. Es wird ein Grenzwert definiert, ab dem ein Pixel als Hintergrund oder als Objekt oder Person erkannt wird. Das Ergebnis gibt häufig eine Binärmaske zurück, in dem der Vor- und Hintergrund als weiß und schwarz dargestellt.
- Morphologische Filter wie Erode, Dilate, Opening und Closing sowie Kantenerkennung werden eingesetzt, um Binärbilder zu verbessern.
- Dilate ist eine Operation, bei welchem die weißen Bereiche eines Binärbildes vergrößert werden. Dabei wird ein Fenster über das Bild gezogen. Falls sich innerhalb dieses Fensters mindestens ein weißer Pixel, wird auch der Mittelpunkt weiß gesetzt. Dadurch werden Lücken geschlossen und benachbarte Bereiche miteinander verbunden
- Erode ist das Gegenstück zu Dilate. Ein Pixel bleibt nur weiß, wenn sich innerhalb des gesamten Fensters ausschließlich weiße Pixel befinden. Der Rest ist schwarz. Dadurch werden kleine Störungen und vergrößerte Bereiche korrigiert.

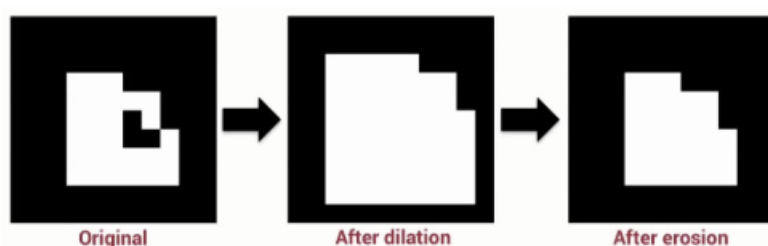


Abbildung 2: Erode & Dilate Bildliche Darstellung

- Das Opening und Closing sind eine Kombination aus Erode und Dilate. Das Opening läuft so ab, dass zuerst ein Erode gemacht wird und anschließend ein Dilate. Das hilft mit der Entfernung kleiner weißen Störungen. Closing ist genau umgekehrt zuerst Dilate dann Erode. Closing wird verwendet, um kleine Unterbrechungen oder schwarze Löcher in einer weißen Silhouette auszufüllen.

4.1.3 Segmentierung

Das Ziel der Segmentierung ist es, bestimmte Bereiche eines Bildes, als Beispiel eine Person, vom Hintergrund zu trennen. Der Black Mirror verwendet neben dem Threshold auch eine Hintergrundsubtraktion. Bei der Hintergrundsubtraktion wird ein stationäres Modell des Hintergrunds erzeugt. Veränderungen im Bild werden als Vordergrund gespeichert.

4.1.4 Zeitliche Glättung

Da die Bilddaten in Echtzeit verarbeitet werden, kann es oft zu Schwankungen, wodurch die Silhouette von Bild zu Bild flackert. Für dieses Problem wird die zeitliche Glättung eingesetzt. Bei der zeitlichen Glättung wird das aktuelle Bild und das vorherige Bild kombiniert. Dadurch entsteht eine stabile Darstellung und reduziert Rauschen.

4.2 Künstliche Intelligenz in der Bildanalyse

4.2.1 Mediapipe Selfie Segmentation

Das Mediapipe Selfie Segmentation ist ein KI basiertes Modell zur Personenerkennung. Die KI wurde mit einer Menge verschiedener Bilder trainiert und kann somit eine Person von einem Hintergrund unterscheiden. Die KI liefert für jeden Pixel eine Wahrscheinlichkeit, ob diese zur Person dazugehört. In Kombination mit Thresholds wird eine Binärmaske erstellt.

4.2.2 Mediapipe Hands

Mediapipe Hands ist ein KI-Modell, welches zur Erkennung von Händen und Fingern verwendet wird. Es werden 21 Landmarkpunkte auf die Gelenke und Fingerkuppen gezeichnet. Diese Punkte erlauben eine präzise Analyse der Handbewegungen und anhand der Landmarkkoordinaten kann eine Handmaske erstellt werden, um zum Beispiel Finger zuverlässig zu erkennen.

4.3 LED-Display-Technik

!!!! Manuel Teil

4.4 Git, Versionsverwaltung und Entwicklungsumgebung

Das GitHub-Projekt wurde ursprünglich zur Dokumentation und Aufgabenverwaltung erstellt. Nach einiger Zeit wurde die Aufgabenverwaltung jedoch vernachlässigt, sodass das Repository schließlich ausschließlich zur Dokumentation sowie zur Archivierung veralteter Codes und Prototypenzeichnungen genutzt wurde. Im weiteren Verlauf wurde ein Rework des Main-Branches durchgeführt. Dabei wurden insbesondere die Übersichtlichkeit und die Struktur grundlegend überarbeitet. Seitdem fungiert ein neuer Branch mit der Bezeichnung „main-branch-rework“ als Hauptbranch. Dieser enthält ein neu strukturiertes und übersichtliches Archiv für veraltete Softwarecodes, Simulationen und Dokumentationen sowie einen Setup-Guide, der beschreibt, wie das Projekt eingerichtet werden kann, um es zu reproduzieren.

5 Analyse und Anforderungen

5.1 Ausgangssituation und Ideenfindung Black Mirror

Die Idee hinter dem Projekt Black Mirror entstand aus einem Video auf Social Media, in welchem ein Spiegel aus vielen mechanischen Sieben-Segmentanzeigen das Spiegelbild der Person darstellte. Dadurch dass die mechanischen Sieben Segment Anzeigen langsam und ungenau die Person spiegelten, entstand die Idee die Person auf einer großen LED-Matrix zu spiegeln.

5.2 Funktionale Anforderungen

Das System ist aufgefordert eine Person zuverlässig zu erkennen und vom Hintergrund klar trennen. Daraufhin soll eine Silhouette erstellt werden, welche die Hände und Finger präzise erfasst, um Bewegungen klar darzustellen.

5.3 Use Cases und Szenarien

Ein typisches Szenario besteht darin, dass sich die Person vor dem Spiegel stellt und sie auf den Led-Matrizen gespiegelt wird. Die Person soll möglichst präzise angezeigt werden, selbst wenn die Person schnellere Bewegungen ausführt mit den Händen oder mit dem ganzen Körper. Verlässt die Person den Aufnahmebereich der Kamera sollten die Led-Matrizen nicht mehr leuchten.

5.4 Systemgrenzen und Annahmen

Der Black Mirror ist für eine stationäre Kamera mit einem definierten Erfassungsbereich ausgelegt. Änderungen der Lichtverhältnisse oder Spiegelungen im Hintergrund könnend dazu führen, dass die Erkennungsqualität oder Hintergrundtrennung suboptimal verläuft. Aufgrund der begrenzten Auflösung der Led-Matrix ist nur eine abstrahierte Darstellung möglich. Das Hand Tracking kann nur zwei Hände erkennen, wodurch Probleme entstehen, sobald mehr als eine Person im Bereich der Kamera erscheint.

6 Evaluierte Lösungsansätze

6.1 Personenerfassung

6.1.1 Bewegungs -und Kantenerkennung

Bei dieser Lösungsmethode wird die Person klassische Bildverarbeitungsoperationen erkannt. Hierbei wird die Person primär mit Bewegungserkennung erkannt. Hierbei wird anfänglich ein Hintergrundmodell mithilfe dem KNN erstellt. Zusätzlich werden Kantenerkennungs -und morphologische Verfahren, für die Stabilisierung der Silhouette und Reduzierung von Störungen.

6.1.2 BodyPix KI-Segmentierung

Im Gegensatz zum Finalcode des Black Mirrors, wird in dieser Lösungsvariante das KI Modell BodyPix verwendet. Das BodyPix-Modell trennt die Person vom Hintergrund, indem es ein neuronales Netzwerk einsetzt. BodyPix erstellt eine Wahrscheinlichkeitsmaske. Aus dieser wird dann eine binäre Silhouette erstellt. Diese Methode benötigt mehr Rechenleistung.

6.1.3 ASCII-Mirror

Eine weitere Methode stellt die Person, erkannt durch Bewegungs- und Kantenerkennung, auf einer ASCII Oberfläche dar. Die Silhouette wird als Matrix von Zeichen z.B. „0“ oder „1“ dargestellt. Diese Darstellungsform zeigt, wie sich komplexe Bildinformationen auf einfache symbolische Matrizen reduzieren lassen.



Abbildung 3: Ausgabe des ASCII-Mirrors

6.2 Led-Signal

6.2.1 Mehr SPIs

Für diese Lösungsmethode werden die Signale Clk, MOSI und CS von verschiedenen SPI-Schnittstellen vom Raspberry Pi 5 verwendet. Da drei Reihen der LED-Matrix mit seriell geschickten Signalen grenzwertig funktioniert, entstand die Idee drei SPI-Schnittstellen zu verwenden. Die erste drei Reihen werden mit SPI 0 angesteuert, die nächsten drei mit SPI 1 und die restlichen drei mit SPI 2. Bei diesem Verfahren entstand das Problem, dass das Ansprechen der anderen SPIs nicht funktionierte. Dazu wird das Mapping für die Led-Anzeige dadurch wesentlich schwerer.

#	NAME		EXPANDED	GPIO
1	3.3v DC Power		DC Power 5v	02
3	GPIO02 (SDA, I ² C1)		DC Power 5v	04
5	GPIO03 (SCL, I ² C1)		Ground	06
7	GPIO04		GPIO14	08
9	Ground		GPIO15	10
11	GPIO17 (SPI1 CE1)		(SPI1 CE0) GPIO18	12
13	GPIO27		Ground	14
15	GPIO22		GPIO23	16
17	3.3v DC Power		GPIO24	18
19	GPIO10 (SPI0 MOSI)		Ground	20
21	GPIO09 (SPI0 MISO)		GPIO25	22
23	GPIO11 (SPI0 SCLK)		(SPI0 CE0) GPIO08	24
25	Ground		(SPI0 CE1) GPIO07	26
27	GPIO00 (SDA0, I ² C)		(SCL0, I ² C) GPIO01	28
29	GPIO05		Ground	30
31	GPIO06		GPIO12	32
33	GPIO13		Ground	34
35	GPIO19 (SPI1 MISO)		(SPI1 CE2) GPIO16	36
37	GPIO26		(SPI1 MOSI) GPIO20	38
39	Ground		(SPI1 CLK) GPIO21	40

Abbildung 4: Raspberry Pi 5 Pinout

6.2.2 Schmitt Trigger Verstärkerschaltung

Ein weiterer Lösungsansatz bestand in der Verwendung einer Verstärkerschaltung mit einem Schmitt Trigger. Ein Schmitt Trigger wird meistens für die Korrektur verrauschter Signale eingesetzt. Der Schmitt Trigger besitzt unterschiedliche Schaltschwellen für steigende/fallende Flanken, wodurch eine höhere Störsicherheit entsteht. Dadurch werden Signale stabilisiert. Für diesen Ansatz wurde eine Schaltung auf Multisim erstellt, welche anschließend simuliert wurde. Die Schmitt Trigger Verstärkerschaltung ist weniger komplex als die Finale Verstärkerschaltung, aber dadurch dass die simulierten Schmitt Trigger pro Stück viel gekostet haben und dazu noch eine zu lange Transitzeit haben, wurde der Finale Lösungsansatz bevorzugt.

6.2.2.1 Schaltung

Die Widerstände parallel mit den Kondensatoren stellen eine ganze Reihe von Led-Matrizen dar. Ein Puls Generator stellt das Signal, dass aus dem Raspberry PI kommt, dar. Dieser wird mit einem Sinus gestört und durch die Reihe von Led-Matrizen durchgeschickt. Das gleiche Signal wandert weiter zum Schmitt Trigger, jedoch wird das Signal danach invertiert, weil der Schmitt Trigger das Signal selbst invertiert. Die Spannungsquelle links neben der Hauptschaltung versorgt den Schmitt Trigger.

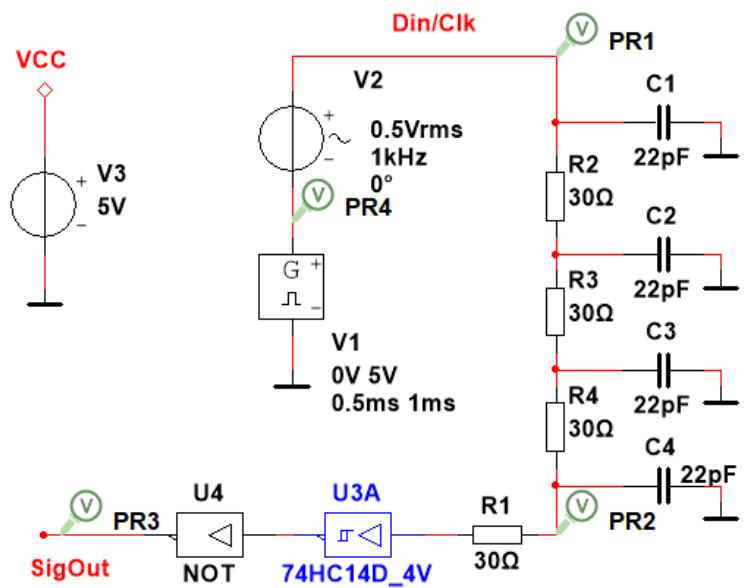


Abbildung 5: Schmid Trigger Schaltung

6.2.2.2 Simulation

Durch die Simulation zeigt sich das Verhalten des Schmitt Triggers. Das Grüne Signal (PR2) zeigt das Verhalten des übertragenen Signals nach einer ganzen Reihe von Led Modulen. Dieses Signal sollte eigentlich ein Rechtecksignal sein, aber dadurch das das Signal durch eine ganze Reihe Led Modulen durchgeht, wird es so gestört, dass es anschließend als Sinus dargestellt wird. Das Blaue Signal (PR4) zeigt das übertragene Signal, nachdem dieser durch den Schmitt Trigger geschickt wird. Die Simulation zeigt die erfolgreiche Korrektur des gestörten Signals.

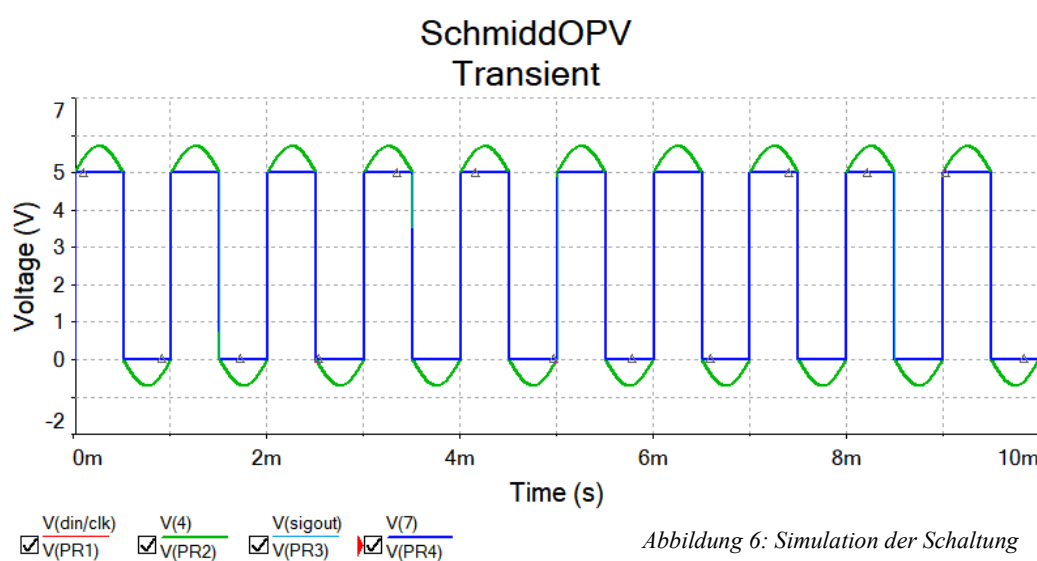


Abbildung 6: Simulation der Schaltung

6.3 Finaler Algorithmus

Nach der Erprobung verschiedener Lösungsmethoden wurde für die Personenerfassung eine KI basierte Segmentierung namens Mediapipe. Mediapipe bietet die Modelle Selfie Segmentation und Handtracking an, welche für den Finalen Code verwendet wurden. Die Lösungsmethoden wie reine Bewegungserkennung und ASCII basierte Darstellung wurden verworfen, da diese entweder eine zu geringe Stabilität oder eine totale Abstraktion der Person aufwiesen.

Im Bereich der Led-Ansteuerung erwies sich, dass die serielle Übertragung des Signals bei einer bestimmten Menge von Led Modulen invalide Darstellungen auf der Led-Matrix zeigte. Die verschiedenen Ansätze, sowie die Verwendung verschiedener SPI-Schnittstellen oder einer Schmitt Trigger Verstärkerschaltung, wurden getestet bzw. simuliert. Es stellte sich heraus, dass eine Kombination aus Signalverstärkung und paralleler Einspeisung der Spannung, die geeignete Lösung für dieses Projekt war.

Die finale basiert also auf einer Kombination aus einer KI-gestützter Bildverarbeitung und einer hardware-basierten optimierten Signalübertragung.

6.4 Case

!!!!!! Sobald es fertig ist

7 Verwendete Technologien

7.1 Programmiersprache und Bibliotheken (Python, OpenCV, NumPy)

Für Personenerfassung und die Led-Ansteuerung wurde hauptsächlich nur die Programmiersprache Python verwendet. Python ist besonders gut für Anwendungen im Themen Bereich der Bildverarbeitung und Künstlicher Intelligenz, da es eine Menge spezialisierten Bibliotheken (packages) zur Verfügung stehen.



Abbildung 7: OpenCv und Python Logo

Für die Kamerasteuerung und Kamerabildverarbeitung wurde die Bibliothek OpenCV verwendet. OpenCv ist eine Bibliothek bekannt für ihre umfangreiche Funktion für Bildverarbeitung, Kameraansteuerung und Filteroptionen.

Da Bilddaten intern als Pixelmatrizen gespeichert werden, ermöglicht die Bibliothek NumPY schnelle mathematische Operationen, um die Bilddaten effizient als Matrizen zu verarbeiten.



Abbildung 8: NumPy Logo

7.2 MediaPipe Selfie Segmentation und Hands

Der Großteil der Personenerfassung besteht aus der KI-basierten Personen und Händeerkennung. Für diese Operationen wurde die Bibliothek Mediapipe eingesetzt, welche ein vortrainiertes Modell zu Verfügung stellt.

Selfie Segmentation ist eines dieser Modelle und wird verwendet, um die Person vom Hintergrund zu trennen. Mediapipe analysiert das Bild und erstellt eine Wahrscheinlichkeitsmaske, die angibt, welche Pixel zur Person gehört und welche zum Hintergrund.

Da Selfie Segmentation für Ganz bzw. Oberkörper trainiert wurde, tut sich die KI schwer Finger und Hände präzise zu erkennen. Deshalb wird das Modell Mediapipe Hands verwendet. Dabei werden 21 sogenannten Landmarkpunkte pro Hand bestimmt. Diese Landmarkpunkte beschreiben die Position der Finger und Gelenke.

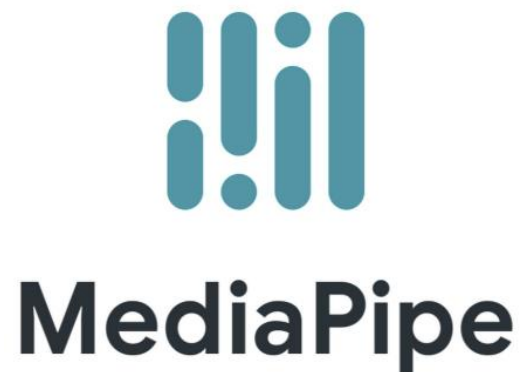


Abbildung 9: Mediapipe Logo

7.3 Bildverarbeitungs- und Filterverfahren

Damit die KI erzeugte Segmentierungsmaske verbessert wird, werden zusätzlich Verfahren für Bildverarbeitung eingesetzt. Zu diesen Verfahren gehören morphologische Operationen wie Dilate und Erode, welche Störungen und kleine Löcher korrigieren. Außerdem werden Verfahren für die Glättung von Kanten und das Reduzieren von Rauschen verwendet. Ebenso wird eine zeitliche Glättung über mehrere Frames verwendet, um eine stabile Darstellung zu erzeugen.

7.4 LED-Ansteuerung

Die Silhouette wird anschließend auf der Led-Matrix angezeigt. Die Kommunikation mit den Led Modulen erfolgt über die Serielle Schnittstelle (SPI), welche der Raspberry Pi über seine GPIO-Pins sendet. Die Silhouette wird auf die Auflösung der Led-Matrix skaliert und pixelweise an die Led Module übertragen.

7.5 Entwicklungswerkzeuge

Es gibt eine Menge an Auswahlmöglichkeiten für die Entwicklung und Verwaltung des Projekts. Der Black Mirror wurde hauptsächlich in PyCharm entwickelt. Zur Verwaltung, Organisation und Dokumentation wurde Git Hub verwendet. Git Hub ist genau für solche Projekte sehr nützlich, da Änderungen am Code nachvollgezogen werden können und verschiedene Stände in der Entwicklung verwaltet werden können.



Abbildung 10 : GitHub Logo



PyCharm

Abbildung 11: PyCharm Logo

8 Systemkonzept und Architektur

8.1 Gesamtüberblick des Systems

Nachdem Einschalten des Black Mirrors erfolgen diese Prozesse, um am Ende ein abstrahiertes Spiegelbild der Person auf der LED-Matrix zu sehen. Die Bilddaten der Kamera werden an den Raspberry Pi weiter versendet. Das Bild wird dann verarbeitet und für die Led-Matrix vorbereitet.



Abbildung 12: Ablaufdiagramm des Systems

8.2 Hardwarearchitektur

8.2.1 Kameramodul

Das Pi Camera Modul 3 Wide NoIR ist ein wichtiger Bestandpunkt des Black Mirrors. Das Modul ist zuständig für Aufnahme der Person in Echtzeit. Das Modul ist klein, weshalb auch ein spezielles Case dafür entwickelt werden musste. Wir haben dieses Kamera Modul ausgewählt, da dieses Modul größere Winkel aufnehmen kann und durch den NoIR-Filter werden die Personen in einem Lila-Ton aufgezeichnet.



Abbildung 13: Pi Camera Modul 3 Wide NoIR

8.2.2 Raspberry Pi

Das Herz des Projekts ist der Raspberry Pi 5. Es verbindet die restlichen Module miteinander und die ganze Software läuft über ihn. Er steuert die Led-Matrizen mit einem kleinen Schieberegister. Er beinhaltet die Software für die Personenerkennung und die Darstellung auf den Led-Matrizen. Wir haben uns für den Raspberry entschieden, da dieser genug Rechenleistung aufweist, damit die Ansteuerung und Personenerfassung gleichzeitig erfolgen können.



Abbildung 14: Raspberry Pi 5

8.2.3 MAX7219

Die LEDs zeigen das abstrahierte Spiegelbild der Person dar. Es werden LED-Matrizen verwendet die jeweils auf einer Platine, auf welcher 4 LED-Matrizen miteinander verbunden werden. Es werden insgesamt 36 dieser Module verwendet, d.h. es sind insgesamt 144 LED-Matrizen. Die LED-Matrizen sind blaufarbig. Ein Modul ist 12,8 cm hoch und 3,2 cm breit. Die LED-Matrix ist 53,6 cm hoch und 29,5 cm breit. Sie werden über die GPIO Pins des Raspberry angesteuert. Der Raspberry Pi schickt ein Clock Signal, Data In und ein Chip Select durch die ganze LED-Matrix.

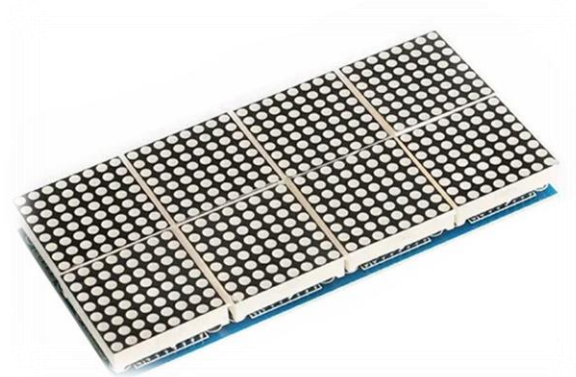


Abbildung 15: MAX7219

8.2.4 Case

Für das Case wurden zum Großteil Holzplatten verwendet. Das Case besteht aus drei Teilen: Der Rahmen, LED-Platte, Back Plate. Für den Rahmen wurden 600x200x18 mm große Holzplatten gekauft. Diese wurden Für die LED-Platte

!!!!!! Sobald das Case fertig gekauft wurde

8.2.5 Weiter Komponente

Unter anderem wurde ein kleines durchsichtiges Case für den Raspberry Pi gekauft. Dieser hat zusätzlich einen Lüfter. Dazu wurden am Raspberry Kühlrippen an einigen Flächen geklebt. Das Netzteil für den Raspberry mit 5V und 5A, welches der Raspberry auch benötigt, und um den Raspberry an einem Monitor anzuschließen, musste ein HDMI-Mini auf HDMI-Kabel gekauft werden. Ebenso wurde ein USB-Stick für den Raspberry gekauft, damit das Betriebssystem für den Pi darauf gespeichert werden kann. Da Das Kameramodul einen 22-Pin Anschluss hat und der Raspberry Pi 5 nur 15-Pin Anschlüsse hat, musste ein spezielles Kabel der die Anschlüsse von 22 Pins auf 15 Pins verringert, ebenfalls gekauft werden.



Abbildung 17: Netzteil für Raspberry Pi



Abbildung 16: Kamera Kabel



Abbildung 18: HDMI zu HDMI-Mini Kabel



Abbildung 19: Case für Raspberry Pi

8.3 Softwarearchitektur

8.3.1 Personenerfassung

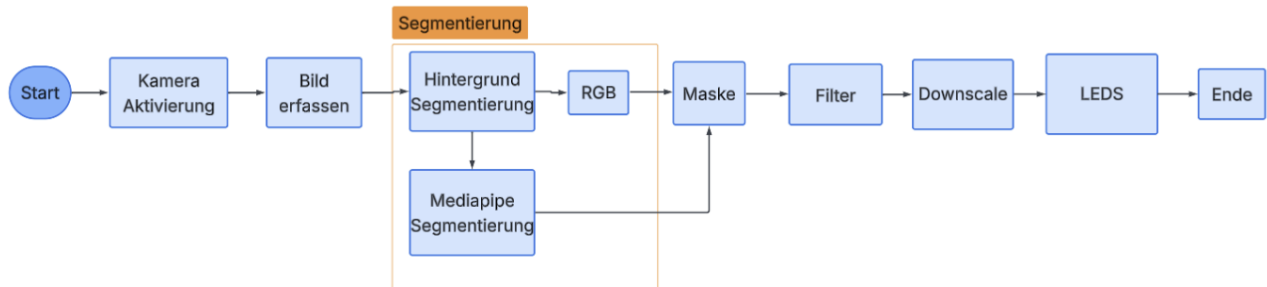


Abbildung 20: Ablaufdiagramm Personenerfassung

Sobald der Prozess der Personenerfassung startet, wird die Kamera aktiviert und erfasst ein Bild. Dieses wird an die Hintergrundsegmentierung sowie an MediaPipe Selfie Segmentation und das Hand-Tracking übergeben, um die Person im Bild zu erkennen.

Nach einer RGB-Umwandlung und Filterkorrektur wird eine Silhouettenmaske erzeugt. Das Ergebnis wird anschließend auf die Auflösung der LED-Matrix skaliert und an die LED-Ansteuerung weitergegeben.

8.3.2 LED-Ansteuerung

Ein von OpenCV bereitgestellter Frame dient als Eingabe und wird zunächst darauf geprüft, ob die Anwendung auf einem Raspberry Pi läuft. Diese plattformabhängige Abfrage ermöglicht es, die Hardwareansteuerung nur bei vorhandener LED-Matrix zu aktivieren. Bei aktiver Hardware wird die LED-Matrix über die SPI-Schnittstelle initialisiert und konfiguriert.

Die eigentliche Verarbeitung folgt den im Diagramm dargestellten Schritten: Das Bild wird zunächst in mehrere horizontale 8-Pixel-Blöcke unterteilt, um die Struktur der LED-Module abzubilden. Anschließend werden ausgewählte Blöcke um 180 Grad gedreht, um unterschiedliche physische Ausrichtungen der Module zu korrigieren. Danach erfolgt die Umwandlung des Farbbildes in ein Graustufenbild, wodurch die Datenmenge reduziert und an die Darstellungsfähigkeiten der Matrix angepasst wird. Das resultierende Bild wird in ein kompatibles PIL-Format konvertiert und schließlich auf der LED-Matrix angezeigt.

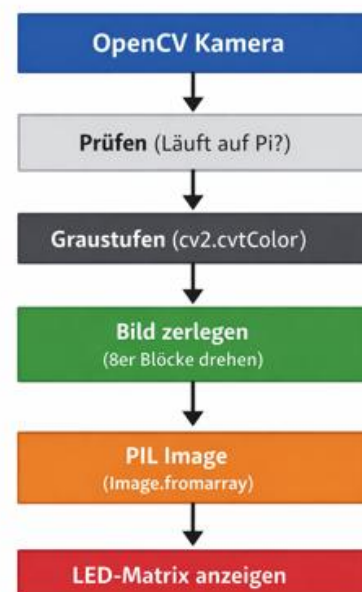


Abbildung 21: Ablaufdiagramm LED-Ansteuerung

9 Personenerfassung Implementierung

Dieser Sourcecode wurde mithilfe von KIs gefertigt.

9.1 Importe und Kameraauswahl

Dieser Abschnitt lädt die Bibliotheken des Systems. OpenCV übernimmt die Bildverarbeitung, NumPy die Arbeit mit Pixelmatrizen und MediaPipe die KI-Modelle für Segmentierung und Handtracking. Das Modul ImagetoMatrix stellt die Verbindung für die Ausgabe an der Led-Matrix

```
import cv2          # OpenCV für Bildverarbeitung und mathematische Matrix-Operationen
import numpy as np   # NumPy für schnelle numerische Berechnungen von Arrays
import mediapipe as mp # MediaPipe für ML-basierte Körper- und Handerkennung
import Software.LED_Matrix.ImagetoMatrix as ImagetoMatrix # Eigenes Modul zur LED-Matrix-
Ansteuerung
```

9.2 Kameraauswahl

Hier wird geprüft, ob das Programm auf einem Raspberry Pi mit PiCamera2 läuft oder auf einem normalen System mit Webcam. Dadurch bleibt die übrige Programmlogik unverändert, obwohl unterschiedliche Hardware verwendet wird.

```
USE_PI = False
try:
    # Versuche die spezifische Raspberry Pi Kamera Bibliothek zu laden
    from picamera2 import Picamera2
    USE_PI = True
    print("PiCamera2 gefunden - benutze Raspberry-Kamera.")
except ImportError:
    # Fallback für die Entwicklung auf normalen Laptops oder PCs
    USE_PI = False
    print("PiCamera2 nicht gefunden - benutze cv2.VideoCapture.")
```

9.3 Kamerainitialisierung & Auflösung

Die Kamera wird mit einer Auflösung von 640 x 480 Pixeln initialisiert. Diese Werte wurden mit Absicht so gewählt. Das Programm soll eine hohe Bildrate haben aber gleichzeitig ohne möglichst viel Latenz. Das Format BGR888 bedeutet, dass drei Farbkanäle mit jeweils 8 Bit verwendet werden, was dem Standardformat von OpenCV entspricht.

```
def init_camera():
    """ Initialisiert die Kamera je nach erkannter Hardware. """
    if USE_PI:
        picam2 = Picamera2() # Auflösung auf 640x480 festlegen
        config = picam2.create_preview_configuration(
            main={"format": "BGR888", "size": (640, 480)}
        )
        picam2.configure(config)
        picam2.start()
        return picam2
    else:
        # Standard OpenCV Methode für allgemeine Webcams
        cap = cv2.VideoCapture(0)
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        return cap
```

9.4 Selfie Segmentation

Mit SelfieSegmentation wird das KI-Modell zur Personenerkennung initialisiert. Der Parameter `model_selection = 1` aktiviert das allgemeinere Modell der KI, welcher für die Ganzkörpersegmentierung zuständig ist. Diese Wahl ist wichtig, da die komplette Person erkannt werden soll.

```
mp_selfie = mp.solutions.selfie_segmentation
# model_selection=1 verwendet das allgemeinere Modell für vollständige Personenerkennung
segmenter = mp_selfie.SelfieSegmentation(model_selection=1)
)
```

9.5 Hand-Tracking

Dieses Modell erkennt Hände und ihre Landmarken. `static_image_mode = False` schaltet in den Videomodus, wodurch erkannte Hände in mehreren Frames verfolgt werden und die Verarbeitung stabiler wird. `min_detection_confidence = 0.5` bestimmt die Sicherheit, ab welche eine Hand als erkannt gilt. `min_tracking_confidence = 0.6` erklärt die Stabilität des Trackings zwischen den Frames, um Sprünge in der Erkennung zu reduzieren.

```
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    static_image_mode=False,          # Videomodus: nutzt Tracking zwischen Frames
    max_num_hands=2,                 # Begrenzung auf zwei Hände spart Rechenzeit
    min_detection_confidence=0.5,     # Schwellwert für die initiale Hand-Erkennung
    min_tracking_confidence=0.6       # Höherer Wert für stabileres Tracking
```

9.6 Hintergrundmodell & Kalibrierungsphase

Dieser Abschnitt erzeugt ein klassisches Hintergrundmodell auf Basis des KNN-Verfahrens. `history = 150` legt fest, wie viele frühere Frames zum Lernen des Hintergrunds berücksichtigt werden. `dist2Threshold = 400` bestimmt die Empfindlichkeit gegenüber Abweichungen vom Hintergrund: kleinere Werte = sensibler, größere Werte = robuster gegen Bildrauschen. `detectShadows = False` deaktiviert die Schattenerkennung, da für die Schwarz-Weiß-Silhouette dies nicht nötig ist.

```
fgbg = cv2.createBackgroundSubtractorKNN(
    history=150,                      # Anzahl vergangener Frames für das Hintergrundmodell
    dist2Threshold=400,               # Empfindlichkeit gegenüber Abweichungen vom Hintergrund
    detectShadows=False               # Schatten werden nicht separat als Vordergrundklasse behandelt
)
calibration_frames = 100
print("Kalibrierung: Bitte den sichtbaren Bereich verlassen. Die Kamera lernt den Hintergrund.")

for i in range(calibration_frames):
    ret, frame = get_frame(cam)
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # Graustufenbild für den Subtractor
    fgbg.apply(gray, learningRate=0.5)             # Hohe Lernrate für schnelles Einlernen des
Hintergrunds
```

9.7 Helligkeitsanpassung

Mit dieser Transformation werden Helligkeit und Kontrast angepasst. `alpha_bright = 1.3` verstärkt den Kontrast, indem die Pixelwerte mit 1.3 multipliziert werden. `beta_bright = 20` addiert zusätzlich eine konstante Aufhellung. Diese Kombination verbessert die Sichtbarkeit der Person bei schwachen Lichtverhältnissen

```
alpha_bright = 1.3          # Kontrast um 30 Prozent erhöhen
beta_bright = 20            # Helligkeit um einen konstanten Wert anheben
frame_enh = cv2.convertScaleAbs(frame, alpha=alpha_bright, beta=beta_bright)
```

9.8 Personensegmentierung

OpenCV arbeitet standardmäßig im BGR-Farbraum, MediaPipe erwartet jedoch RGB, weshalb die Umwandlung mit `cvtColor` erforderlich ist. Mit dem Threshold 0.4 wird entschieden, ab welcher Wahrscheinlichkeit ein Pixel zur Person gehört. Ein niedrigerer Schwellenwert würde mehr Hintergrund mitnehmen, ein höherer Wert könnte feine Körperbereiche abschneiden. Durch `astype(np.uint8)` und die Multiplikation mit 255 entsteht eine Binärmaske im OpenCV-Format mit den Werten 0 und 255.

```
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # MediaPipe erwartet RGB statt BGR
res = segmenter.process(rgb)
# Ab 40 Prozent Wahrscheinlichkeit wird ein Pixel als Teil der Person gewertet
ki_mask = (res.segmentation_mask > 0.4).astype(np.uint8) * 255 # Binärmaske mit Werten 0 oder 255
```

9.9 Handerkennung & Umrechnung der Landmarken

Hier wird das Handmodell auf das aktuelle Bild angewendet. Zunächst wird eine leere Maske mit derselben Größe wie die Körpermaske erzeugt. Durch die Multiplikation mit w und h werden sie in Pixelkoordinaten umgerechnet. Nur somit kann anschließend ein Polygon erstellt werden.

```
hand_res = hands.process(rgb)          # Zweiter KI-Schritt: nur auf Hände spezialisiert
hand_mask = np.zeros_like(ki_mask)     # Leere Maske für die spätere Hand-Silhouette

if hand_res.multi_hand_landmarks:
    h, w, _ = frame.shape
    for handLms in hand_res.multi_hand_landmarks:
        hand_points = []
        for lm in handLms.landmark:
            cx = int(lm.x * w)          # x von normiertem Wert in Pixel umrechnen
            cy = int(lm.y * h)          # y von normiertem Wert in Pixel umrechnen
            hand_points.append([cx, cy])
```

9.10 Handflächenaufbau

Hier wird aus ausgewählten Landmarken ein Polygon der Handfläche aufgebaut. Die Parameter 0, 1, 5, 9, 13 und 17 stellen Handgelenk und Fingerbasen dar und bilden die innere Struktur der Handfläche gut ab. fillPoly füllt diese Fläche mit dem Wert 255, sodass ein geschlossener weißer Bereich entsteht.

```
# Handfläche als geschlossenes Polygon aufbauen
palm_indices = [0, 1, 5, 9, 13, 17] # Landmarken der inneren Handfläche
palm_pts = np.array([hand_points[i] for i in palm_indices], dtype=np.int32)
cv2.fillPoly(hand_mask, [palm_pts], 255) # Handfläche vollständig weiß füllen
```

9.11 Finger, Verbindungen und Gelenke

Die Finger werden als Linien zwischen bestimmten Landmarken gezeichnet. Der Parameter `finger_thickness = 18` ist groß gewählt, da die Finger auch nach der Verkleinerung auf 32 x 48 Pixel sichtbar bleiben müssen. Der Kreis an jedem Landmark besitzt mit `finger_thickness // 2` einen passenden Radius und glättet das Gelenk, sodass eine zusammenhaltende Handform entsteht.

```
finger_thickness = 18 # Dicke Linien, damit Finger nach dem Downscaling sichtbar bleiben
for connection in connections:
    pt1 = tuple(hand_points[connection[0]])
    pt2 = tuple(hand_points[connection[1]])
    cv2.line(hand_mask, pt1, pt2, 255, thickness=finger_thickness)

for point in hand_points:
    cv2.circle(hand_mask, tuple(point), finger_thickness // 2, 255, -1) # Kreise glätten die
    Gelenke
```

9.12 Morphologische Bearbeitung der Maske

Diese Filter formen die gezeichnete Hand zu einer Naturellen Silhouette um. Der 5 x 5 Kernel Dilatation entfernt kleine Lücken und verbindet benachbarte weiße Bereiche. `iterations = 1` reicht aus, um eine stabile Form zu bilden, ohne sie stark zu vergrößern. Der Gauß-Filter mit Kernel 9 x 9 glättet harte Kanten. Anschließend stellt `threshold` mit dem Wert 120 wieder eine Binärmaske her. Die abschließende Erosion mit einem 3 x 3 Kernel verkleinert die zuvor ausgedehnte Hand leicht.

```
kernel_medium = np.ones((5, 5), np.uint8) # Mittlerer Kernel zum Schließen kleiner Lücken
hand_mask = cv2.dilate(hand_mask, kernel_medium, iterations=1) # Vergrößert weiße Bereiche
leicht

hand_mask = cv2.GaussianBlur(hand_mask, (9, 9), 0) # Macht Kanten weicher und
organischer
_, hand_mask = cv2.threshold(hand_mask, 120, 255, cv2.THRESH_BINARY) # Wieder in Schwarz-Weiß
umwandeln

kernel_small = np.ones((3, 3), np.uint8) # Kleiner Kernel für minimale Rückkorrektur
hand_mask = cv2.erode(hand_mask, kernel_small, iterations=1) # Schrumpft die Hand leicht auf
realistischere Größe
```

9.13 Zeitliche Glättung

Dieser Abschnitt reduziert Flackern zwischen aufeinanderfolgenden Frames. `addWeighted` mischt die vorherige und die aktuelle Handmaske. `alpha_hand = 0.35` bedeutet, dass 35 Prozent der alten und 65 Prozent der neuen Maske das neue Bild bilden. Dadurch wird die Darstellung geglättet, ohne viel Delay. Der anschließende Threshold 140 wandelt die Grauwertmaske wieder in eine saubere Binärmasken um.

```
alpha_hand = 0.35 # Anteil der alten Handmaske für zeitliche Stabilisierung

if prev_hand_mask is None:
    prev_hand_mask = hand_mask.copy()
else:
    hand_blended = cv2.addWeighted(
        prev_hand_mask, alpha_hand,      # Vorherige Maske wirkt mit 35 Prozent nach
        hand_mask, 1 - alpha_hand, 0    # Aktuelle Maske dominiert mit 65 Prozent
    )
    _, hand_blended = cv2.threshold(hand_blended, 140, 255, cv2.THRESH_BINARY)

# Wieder scharf binarisieren
```

9.14 Maskenfusion & Bereinigung

Mit `bitwise_or` werden die Personensegmentierung und die Handmaske zusammengeführt. Ein Pixel bleibt weiß, wenn er in einer der beiden Masken aktiv ist. Dadurch werden die detaillierten Hände in die Ganzkörpersilhouette integriert. Der 3 x 3 Kernel ist klein gewählt, damit nur feine Fehler korrigiert werden. `iterations = 1` verhindert, dass benachbarte Formen ungewollt zusammenwachsen.

```
ki_mask = cv2.bitwise_or(ki_mask, hand_mask) # Pixel bleibt weiß, wenn Person oder Hand erkannt
wurde

kernel_denoise = np.ones((3, 3), np.uint8) # Kleiner Kernel für sanfte Nachbearbeitung
ki_mask = cv2.morphologyEx(
    ki_mask,
    cv2.MORPH_CLOSE,      # Schließt kleine schwarze Löcher innerhalb weißer Flächen
    kernel_denoise,
    iterations=1
)
```


9.15 Konturen & finale Glättung

Der Bilateral-Filter reduziert kleine Unregelmäßigkeiten in der Maske. Der Parameter 5 definiert die Größe des lokalen Nachbarschaftsbereichs. Die Werte 50 und 50 bestimmen, wie stark Helligkeitsunterschiede und räumliche Abstände in die Glättung mitgezählt werden. Der erneute Threshold 140 stellt sicher, dass das Bild wieder eine reine Schwarz-Weiß-Maske ist.

```
final_mask = cv2.bilateralFilter(ki_mask, 5, 50, 50) # Rauschen reduzieren, Kanten möglichst
erhalten
_, final_mask = cv2.threshold(final_mask, 140, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(final_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
mask_filled = np.zeros_like(final_mask)

min_area = 200 # Sehr kleine Flächen gelten als Rauschen und werden verworfen
for contour in contours:
    if cv2.contourArea(contour) > min_area:
        cv2.drawContours(mask_filled, [contour], -1, 255, thickness=cv2.FILLED) # Kontur
vollständig füllen

alpha = 0.25 # Leichtere zeitliche Glättung für die gesamte Silhouette
blended = cv2.addWeighted(prev_mask, alpha, final_mask, 1 - alpha, 0)
```

Mit findContours werden zusammenhängende weiße Bereiche der Maske erkannt.

RETR_EXTERNAL sorgt dafür, dass nur die äußeren Konturen betrachtet werden. min_area = 200 filtert sehr kleine Störflächen, die meist durch Rauschen entstehen. drawContours mit thickness = cv2.FILLED füllt die verbleibenden Konturen vollständig aus und erzeugt damit eine kompakte Form.

9.16 Ausgabe

Da Kamera und LED-Matrix unterschiedliche Seitenverhältnisse besitzen, wird zunächst ein passender Bildausschnitt berechnet. $\text{aspect} = 32 / 48$ entspricht exakt dem Seitenverhältnis der LED-Matrix. Ist das Kamerabild im Verhältnis breiter, wird die Breite reduziert. Andernfalls die Höhe. So bleibt die Silhouette proportional und wird beim Verkleinern nicht verzerrt.

```
h, w = out_full.shape[:2]
aspect = 32 / 48 # Seitenverhältnis der LED-Matrix

if w / h > aspect:
    crop_w = int(h * aspect) # Bild ist zu breit -> Breite reduzieren
    crop_h = h
else:
    crop_w = w
    crop_h = int(w / aspect) # Bild ist zu hoch -> Höhe reduzieren

cropped = cv2.getRectSubPix(out_full, (crop_w, crop_h), (w // 2, h // 2)) # Mittigen Ausschnitt
extrahieren
out_small = cv2.resize(cropped, (32, 48), interpolation=cv2.INTER_AREA) # Auf Matrixauflösung
verkleinern
ImagetoMatrix.drawImage(out_small) # Bild an LED-Matrix
senden
```

10 Led-Ansteuerung Implementierung

10.1 Importieren der Bibliotheken

Zu Beginn werden alle benötigten Bibliotheken eingebunden: sys zur Plattformerkenung, cv2 (OpenCV) für die Bildverarbeitung, numpy für Array-Operationen und PIL zur späteren Bildkonvertierung. Diese Kombination ermöglicht die Verarbeitung und Umwandlung von Kamerabildern in ein für die LED-Matrix geeignetes Format.

```
import sys
import cv2
import numpy as np
from PIL import Image
```

Abbildung 22: Bibliotheken Import

10.2 Plattformerkenung

Hier wird überprüft, ob das Programm auf einem Linux-System läuft, was als Indikator für den Raspberry Pi dient. Diese Abfrage ist wichtig, um zwischen Entwicklungsumgebung und Zielhardware zu unterscheiden.

```
RUN_ON_PI = sys.platform.startswith("linux")
```

10.3 Initialisierung der LED-Matrix (nur auf Raspberry Pi)

```
if RUN_ON_PI:
    from luma.led_matrix.device import max7219
    from luma.core.interface.serial import spi, noop

    serial = spi(port=0, device=0, gpio=noop())
    device = max7219(serial, block_orientation=-90, width=32, height=48)
    device.contrast(10)
else:
    device = None # Platzhalter ohne Hardware
```

Abbildung 23: Initialisierung

Wenn das Programm auf dem Raspberry Pi läuft, wird die LED-Matrix über die SPI-Schnittstelle initialisiert. Dabei werden Auflösung, Ausrichtung und Helligkeit festgelegt. Andernfalls wird device auf None gesetzt, sodass der restliche Code ohne Hardware ausgeführt werden kann.

10.4 Hauptfunktion zur Bilddarstellung

```
def drawImage(frame):  
    # Auf dem Laptop: einfach nichts machen  
    if device is None:  
        return
```

Abbildung 24: drawImage Funktion

Diese Funktion übernimmt ein Kamerabild (frame) und führt alle weiteren Verarbeitungsschritte durch, bis es auf der LED-Matrix angezeigt wird.

Falls keine LED-Matrix vorhanden ist (z. B. beim Testen auf einem Laptop), wird die Funktion sofort beendet. Dadurch wird verhindert, dass hardwareabhängiger Code ausgeführt wird.

10.5 Aufteilen des Bildes in Blöcke

Das Bild wird in horizontale Segmente von jeweils 8 Pixeln Höhe unterteilt. Diese Struktur entspricht den einzelnen 8×8-Modulen der LED-Matrix.

```
erste = frame[0:8, :]  
zweite = frame[8:16, :]  
dritte = frame[16:24, :]  
vierte = frame[24:32, :]  
funfte = frame[32:40, :]  
sechste = frame[40:48, :]
```

10.6 Drehen einzelner Blöcke

Bestimmte Blöcke werden um 180 Grad gedreht (`rot90(..., 2)`), um unterschiedliche physische Ausrichtungen der LED-Module auszugleichen. Nicht alle Blöcke werden gedreht, da einige bereits korrekt orientiert sind.

```
frame[0:8, :] = np.rot90(erste, k: 2)  
# frame[8:16, :] = np.rot90(zweite, 2)  
frame[16:24, :] = np.rot90(dritte, k: 2)  
# frame[24:32, :] = np.rot90(vierte, 2)  
frame[32:40, :] = np.rot90(funfte, k: 2)  
# frame[40:48, :] = np.rot90(sechste, 2)
```

10.7 Umwandlung in Graustufen & Convert to PIL-Image

Das Farbbild wird in ein Graustufenbild umgewandelt. Dadurch wird die Datenmenge reduziert und das Bild an die Möglichkeiten der LED-Matrix angepasst.

Das NumPy-Array wird in ein PIL-Image umgewandelt, da die verwendete Bibliothek dieses Format benötigt. Anschließend wird das Bild in den Modus der LED-Matrix konvertiert.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
img = Image.fromarray(gray)
img = img.convert(device.mode)
```

10.8 Finale Anzeige

```
device.display(img)
```

Das fertige Bild wird an die LED-Matrix übertragen und dargestellt. Dies ist der letzte Schritt der Verarbeitungskette.

10.9 Anmerkung

Dies ist der Code der bei der Vereinfachten Version des Projektes getestet benutzt und unter anderem beim Tag der Offenen Tür vorgezeigt wurde. Die Vereinfachte Version bestand nur aus 6 4er Led Matrix Blöcken. Da der vorausgesetzte Fortschritt zum Schreiben des Finalen Codes zum Zeitpunkt dieser Dokumentation noch nicht erreicht wurde dient dieser als simple Veranschaulichung der Grundidee.

11 Elektronische Umsetzung der Led-Matrix

11.1 Verbindungsplatine

Die Verbindung zwischen den Led Modulen erfolgt durch sogenannte Zwischenplatten/Verbindungsplatten. Diese ersetzt die ursprünglichen U-förmige Pin-Header, da die Zwischenplatten ermöglichen eine deutlich stabilere Verbindung zwischen den Modulen. Die Bauform der Platine wurde so entworfen, dass die Led-Module enger zueinander positioniert, wodurch ein flüssiges Gesamtbild entsteht. Zusätzlich verbessert die feste Lötverbindung die Kontaktqualität und reduziert mögliche Kontaktprobleme

11.1.1 Layout

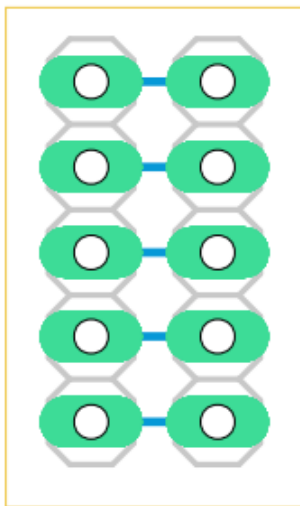


Abbildung 25: Layout Verbindungsplatine

11.1.2 Platine

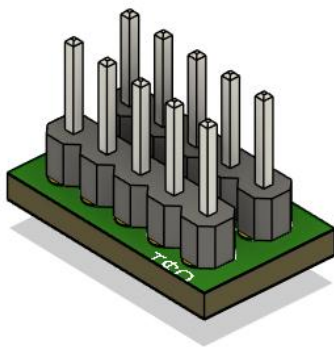


Abbildung 26: 3D Darstellung Verbindungsplatine

11.2 Verstärkerplatine

Die fünf Signale, welche an jedem Led Modul serielle geschickt werden, müssen möglichst rauschfrei nach jedem Modul aussehen. Das Clk Signal vom Raspberry Pi soll gleich aussehen wie das Clk Signal nach dem vierten Led-Modul. Jedoch ändert sich das Clk sowie das Data In Signal schon nach dem vierten Modul und wird unerkennbar zum Anfangssignale.

Das folgende Bild beschreibt das Clk Signal nach dem vierten Led Modul. Im Bild erkennt man wie das eigentliche Clk Signal (Bild links) nach der vierten Reihe sich schon in einem Sinus verwandelt hat. Das ist schlecht für die Led Module, da beispielsweise das fünfte Led Modul ein verwaschenes Rechtecksignal. Dadurch dass die

11.2.1 OPA354AIDBVT

Für die Signalaufbereitung für die Led-Matrix wurde ein Operationsverstärker (OPA354AIDBVT) ausgewählt. Ein OPV ist ein analoges Bauteil, welches die Spannungsdifferenz zwischen zwei Eingängen verstärkt. Ein OPV wird häufig in der Form einer Schaltung als Stabilisierung von Signalen oder Pegelverluste auszugleichen.

Der OPA354 wurde gewählt, da dieser besonders für Anwendungen mit hoher Bandbreite und schnellen Signalflanken geeignet ist. So ein OPV ist sehr wichtig bei serieller Ansteuerung von Led-Matrizen, da ein Signal sowie Clk auch nach mehreren Led Modulen die gleiche Form behalten.

Eigenschaften OPA354:

- Versorgungsspannung: 2,5V bis 5,5V
- Gain-Bandwidth-Product: ca .100 MHz
- Slew Rate: ca. 150 V/ μ s
- Rail-to-Rail Ausgang
- Geringe Offsetspannung

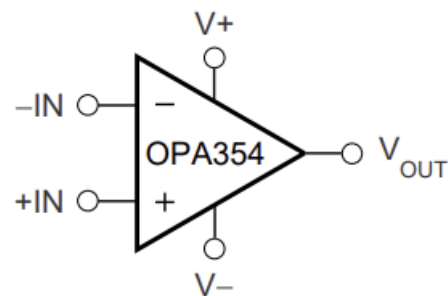


Abbildung 27: OPA354 Schaltzeichen

Durch die hohe Slew-Rate kann der OPV schnelle Signaländerung sauber übertragen. Dadurch dass der OPV eine große Bandbreite hat, treten auch keine starken Signalverzerrungen vor. Der OPV wurde in einer nicht-invertierenden Verstärkerschaltung eingesetzt, um das Signal nach mehreren Led Modulen aufzubereiten und eine passende Signalform wiederherstellen.

11.2.2 Schaltung auf Multisim

Ziel dieser Schaltung ist es das übertragene Signal zu verstärken und die Signalform möglichst unverändert zu übertragen. Bei vielen Led-Matrizen oder langen Leitungen treten parasitäre Kapazitäten auf, welche das Signal verzerren.

Der OPV wird als nicht-invertierender Verstärker beschalten. Die Verstärkung wird durch das Verhältnis des Widerstands R7 in der Rückkopplung zum Spannungsteiler, welches durch die zwei Widerstände R1 und R5 erreicht werden. Der Kondensator C5 wirkt als Abblockkondensator, der hochfrequente Störungen unterdrückt, und dient zur Stabilisierung der Versorgungsspannung. Der Spannungsteiler wird für die Definition der High und Low Pegel eingesetzt. Das ist bei Rechtecksignalen besonders wichtig, sodass das Signal symmetrisch um ein definiertes Mittel verarbeitet werden kann. Auf der Eingangsseite wurde ein RC-Netzwerk modelliert, welches die parasitären Eigenschaften einer Led-Reihe darstellen. Messpunkte PR1, PR2 und PR3 ermöglichen die Simulation vor und nach der Übertragung.

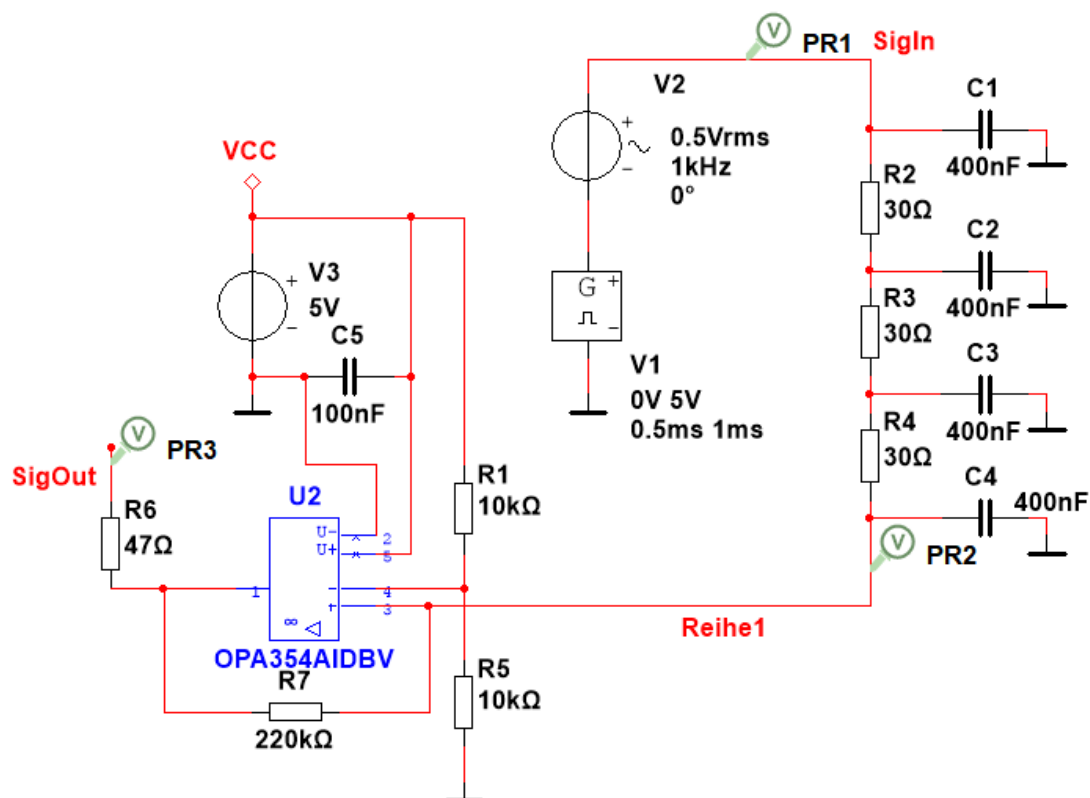


Abbildung 28: OPV-Verstärkerschaltung

11.2.3 Simulation der Verstärkerschaltung

Die Schaltung wurde durch die Transient-Simulation analysiert. Durch diese Analyse wurde die Wirksamkeit der Signal Korrektur untersucht. Das Diagramm zeigt drei Signalverläufe. Das Rote Signal SIGIN stellt ein Rechtecksignal, welches mit einem Sinus gestört wurde, dar. Das grüne Signal REIHE1 zeigt, wie sich das Eingangssignal durch das RC-Netzwerk verzerrt und deutlich verändert. Flankenveränderungen und Verzögerungen treten ins Besondere da auf.

Das Ausgangssignal der OPV-Schaltung zeigt ein ideales Rechteck Signal. Das Signal weist höhere Stabilität Verzerrung und geringere Verzerrung auf. Die Simulation zeigt, dass bei der Ansteuerung langer Led-Ketten eine geeignete Schaltung zur Korrektur des Signals.

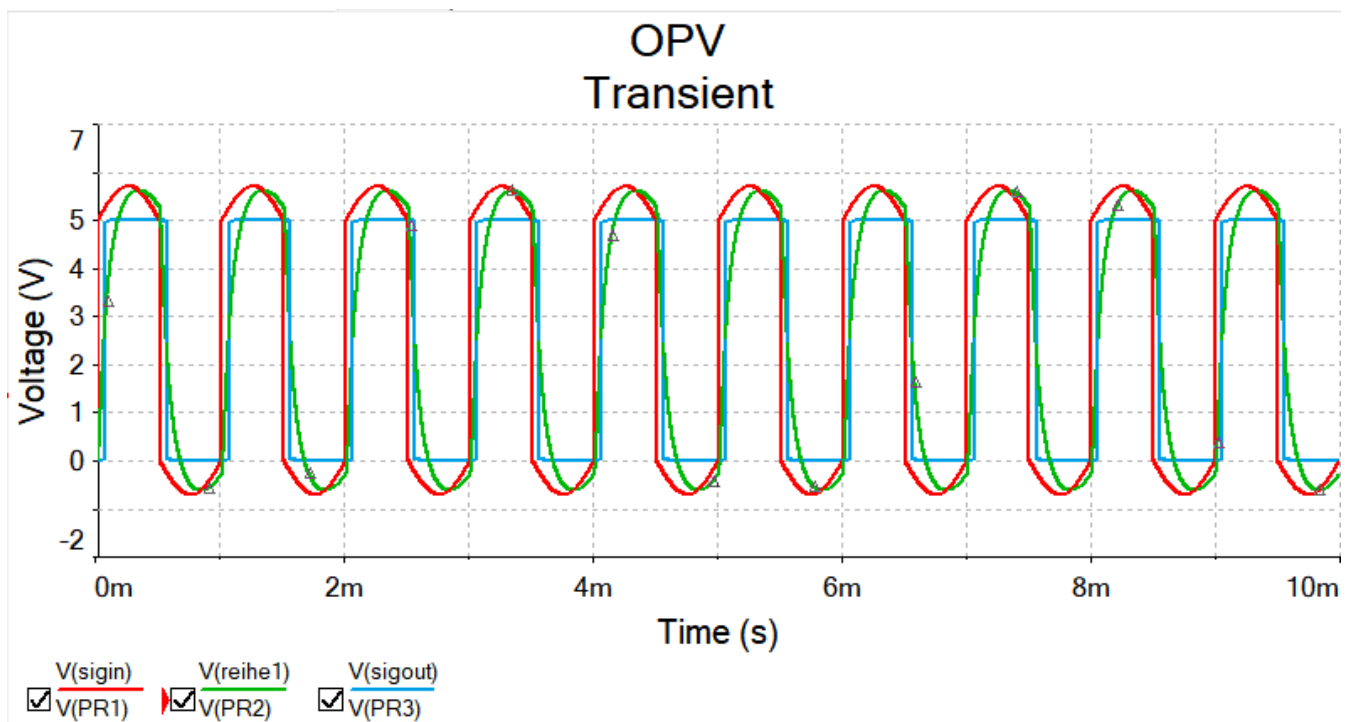


Abbildung 29: OPV-Verstärkerschaltung Simulation

11.2.4 Layout

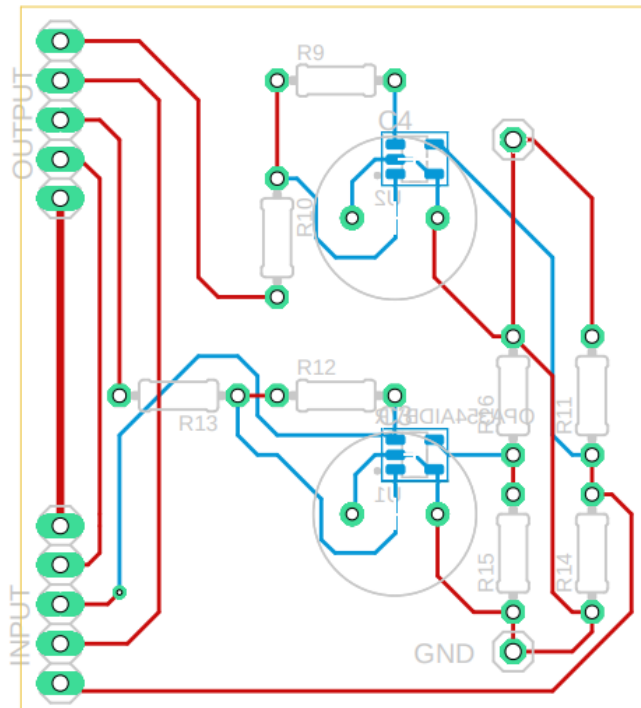


Abbildung 30: Layout der Verstärkerplatine

11.2.5 Platine

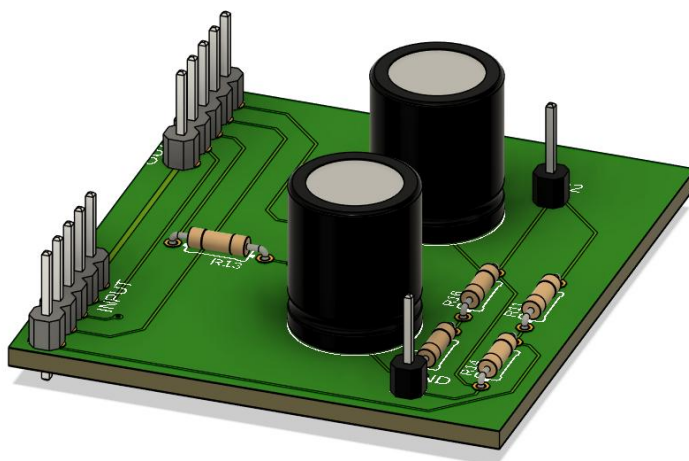


Abbildung 31: 3D Darstellung der Verstärkerplatine

11.3 Levelshifterplatine

Zur stabilen Signalübertragung zwischen dem Raspberry und der Led-Matrix wurde eine Levelshifterplatine eingesetzt. Dadurch, dass der Raspberry Pi mit einer Logikspannung von 3,3 V arbeitet, die Led-Matrix jedoch 5 V benötigt, ist eine Pegelanpassung durch den Level-Shifter erforderlich. Der Level-Shifter wandelt das Signal vom Raspberry auf eine geeignete Spannung für die Led-Matrix, sodass eine stabile Signalübertragung ermöglicht wird. Zusätzlich verbessert die Platine die Signalqualität, indem sie Signalverzerrungen durch lange Leitungen mindert.

11.3.1 SN74AHCT125N

Als zentraler Baustein der Platine wurde der Bus-Buffer SN74AHCT125N verwendet. Dieser ermöglicht eine schnelle Pegelanpassung von 3,3 V auf 5 V. Der SN74AHCT125N eignet sich besonders für hohe Datenraten, wie bei der Ansteuerung von Led-Matrizen.

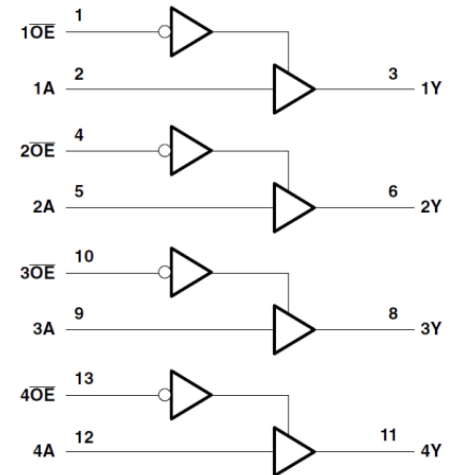


Abbildung 32: Schaltzeichen des Levelshifters

11.3.2 Schaltung auf Multisim

Die simulierte Schaltung besteht aus einer 5V Versorgung, einem Pulsgenerator und einem Level-Shifter. Die Spannungsquelle V2 legt ein Spannungsniveau von 5 V des Ausgangssignals vom Level-Shifter fest. Der Pulsgenerator V1 erstellt das Eingangssignal. Dieser liefert ein Rechtecksignal von 0 V bis 3,3 V und stellt das Ausgangssignal vom Raspberry Pi dar. Dieses Signal wird an den Level-Shifter übertragen. Neben den Eingangs- und Ausgangssignal gibt es noch den OE-Pin (Output Enable). Dieser steuert, ob der Ausgang aktiv ist oder sich in den hochohmigen Zustand befindet. In der Schaltung ist der Ausgang dauerhaft auf aktiv. Am Ausgangspunkt SigOut wird das vom Baustein ausgegebene Signal gemessen. Da der Level-Shifter mit 5 V versorgt wird, ändert sich das Spannungsniveau von 3,3 V auf 5 V.

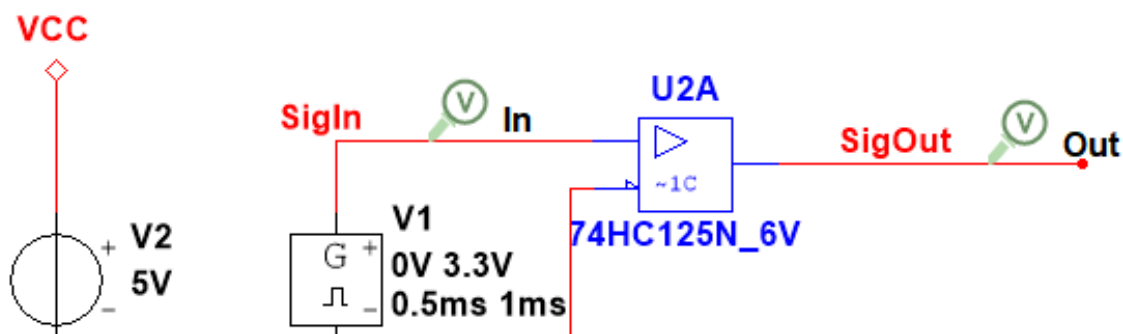


Abbildung 33: Levelshifter-Schaltbild

11.3.3 Simulation

Zur Analyse der Schaltung wurde eine Transienten-Simulation durchgeführt. Hierbei wurde das Eingangssignal des Pulsgenerators mit dem Ausgangssignal des Level-Shifters verglichen. Das Signal *Sigin* erreicht, wie in der Darstellung zu sehen ist, eine Spannung von 3,3 V. Gleichzeitig ist zu erkennen, wie das Ausgangssignal *SigOut* eine deutlich höhere Spannung erreicht. Da in der verwendeten Schaltung kein identisches Modell des realen Bausteins verfügbar war, wurde ein ähnliches Modell verwendet. Dieses erzeugt im Gegensatz zum echten Baustein ein Spannungsniveau von 6 V. Das reale Verhalten des Level-Shifters erzeugt jedoch eine Spannung von 5 V. Trotzdem zeigt die Simulation die gewünschte Funktion: Das 3,3 V Eingangssignal wird durch den Level-Shifter auf ein höheres Ausgangssignal umgesetzt.

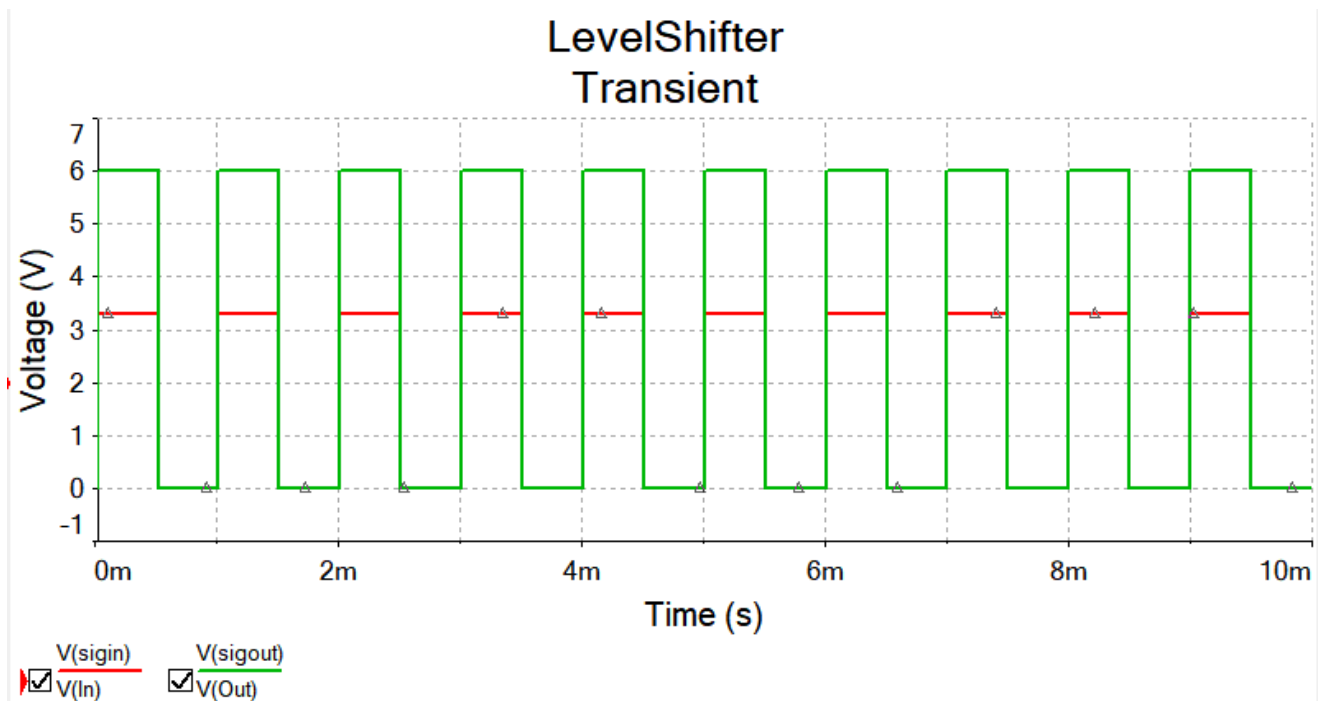


Abbildung 34: Simulation der Levelshifterschaltung

11.3.4 Layout

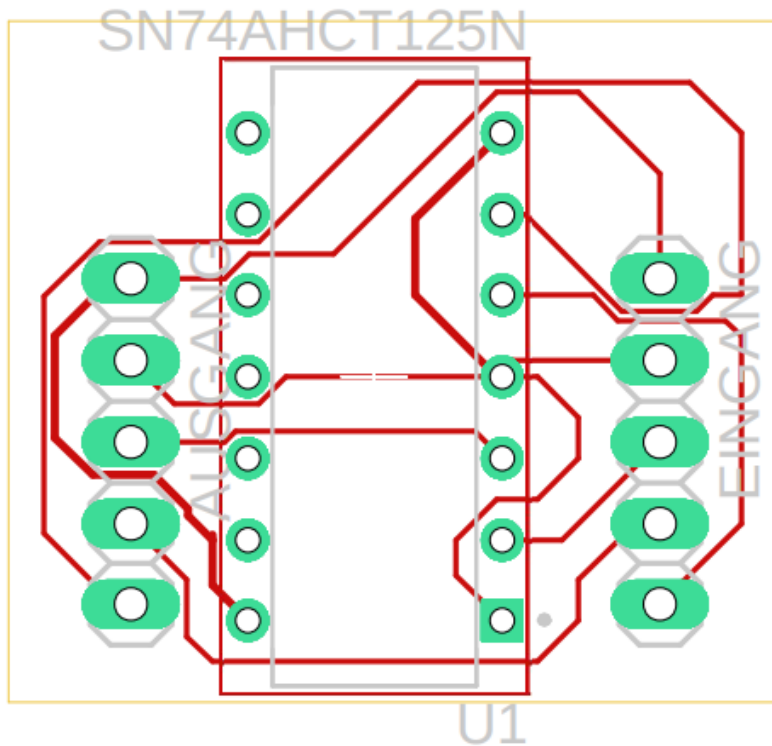


Abbildung 35: Layout der Levelshifterplatine

11.3.5 Platine

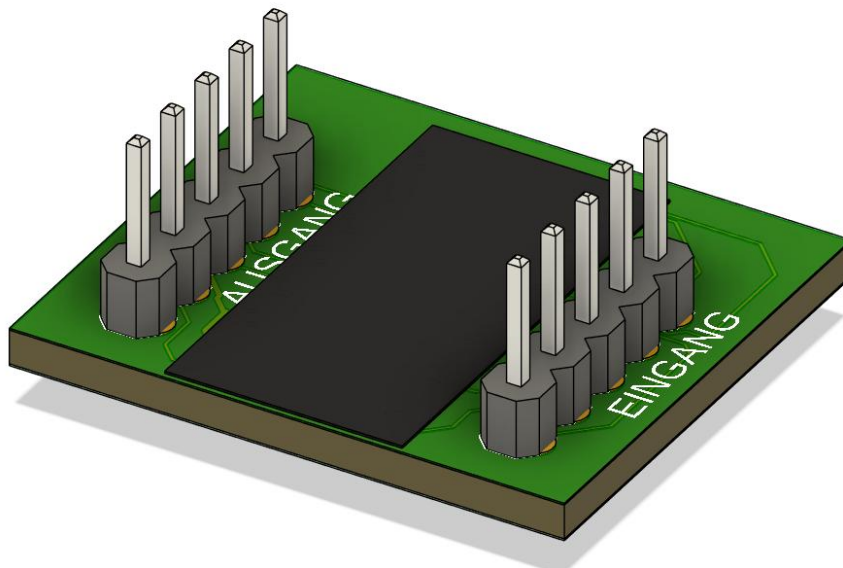


Abbildung 36: 3D Darstellung der Levelshifterplatine

12 Mechanischer Aufbau und Design

12.1 Gehäusekonzept und Spiegelaufbau

12.2 Montage von Kamera und LED-Matrix

12.3 Kabelmanagement und Sicherheit

12.4 Designüberlegungen

!!!!!!!!!!

13 Tests und Evaluierung

Die Personenerfassung, sowie die Led-Ansteuerung und die Signalübertragung wurden nach jedem Meilenstein untersucht. Teilweise getrennt und teilweise wurde das System miteinander verbunden und anschließend untersucht auf Zuverlässigkeit, Leistungsfähigkeit und Grenzen. Die einzelnen Tests wurden unter verschiedenen Bedingungen getestet.

Gesamtbild

Auf dem Bild wird der Raspberry PI mit einem Bildschirm per HDMI zu HDMI Mini Kabel verbunden. Der Raspberry ist mit einem Lan Kabel im gleichen Netz wie der Laptop verbunden. Die Led-Matrix wird von einem Netzteil mit 5V gespeist. Die Led-Matrix steht anschließend mit dem Raspberry noch in Verbindung. Die Kamera klebt an der unteren Rechten Ecke vom Monitor, da die Kamera stationär sein muss und noch keine Hülle gedruckt wurde.

13.1 Teststrategie und Testumgebung

13.1.1 Personenerfassung

Die Personenerfassung die meiste Zeit auf dem Laptop getestet, um neue Funktionen schnell entwickeln und überprüfen zu können. Algorithmen wie die Segmentierung und das Handtracking konnten somit effizient mit der eingebauten Kamera des Laptops angepasst und optimiert werden. Nach der erfolgreichen Umsetzung eines Meilensteins wurde die neue Funktion auf den Raspberry übertragen und unter realen Bedienungen getestet. Dadurch konnte sichergestellt werden, dass diese Funktion auch auf dem Raspberry zuverlässig verläuft. Da der Raspberry das Betriebssystem von Linux verwendet und der Laptop das Betriebssystem Windows verwendet, wurden die entsprechenden Bibliotheken heruntergeladen.

Bild Silhouette

13.1.2 Led-Ansteuerung

13.1.3 Signalübertragung

Die Signalübertragung zwischen dem Raspberry Pi und LED-Matrix wurde besonders viel untersucht, da im Verlauf des Projekts oft Probleme auftraten. Insbesondere bei größeren Matrixgrößen kam es ohne die zusätzlichen Platinen zu Signalstörungen. Durch den Einsatz von Verstärker- und Levelshifterplatinen würde sich die Signalübertragung stabilisieren.

13.2 Funktionstests

13.2.1 Personenerfassung

Für die Funktionstests wurde die Personenerfassung gezielt getestet und anschließend optimiert. Hierbei wurden verschiedene Parameter für Threshold, Filter und morphologische Operationen durchgehend geändert und angepasst. Zusätzlich wurden unterschiedliche Modelle der Mediapipe-Modelle erkundet, um die Erkennungsgenauigkeit zu verbessern. Die Ergebnisse zeigen, dass die Personenerfassung unter guten Bedingungen eine zuverlässige Maske erstellt und unter schlechten Lichtverhältnissen und gespiegelten Hintergrund die Maske an Genauigkeit verliert

13.2.2 Led-Ansteuerung

13.3 Performance-Messungen

13.3.1 Personenerfassung

Damit die Performance bewertet werden kann wurde die Geschwindigkeit der Personenerfassung untersucht. Es stellte sich heraus, dass die Bildrate auf dem Raspberry Pi einen deutlich höheren Wert erreichte als die Bildrate am Laptop. Dennoch arbeitete das System ausreichend schnell auf dem Laptop, um eine Echtzeitdarstellung zu ermöglichen. Bei schnellen Bewegungen und komplexen Szenen traten leichte Verzögerungen bei beide Systemen auf.

13.3.2 Led-Ansteuerung

13.3.3 Signalübertragung

Im Rahmen der Performance-Messung wurde auch die Signalübertragung der LED-Matrix untersucht. Dabei stellte sich heraus, dass die Ansteuerung mit einer kleinen Matrix aus sechs LED-Modulen stabil und ohne Verzögerung funktioniert. Die Darstellung der Silhouette verliert an Genauigkeit, da weniger Pixel vorhanden ist. Die Darstellung einer Silhouette erfolgte flüssig und die Signalqualität war ausreichend. Bei einer Erweiterung der Matrix auf die geplante Größe von 16x9 LED-Matrizen traten jedoch deutlich einige Probleme auf. Ohne die zusätzlichen Platinen funktionieren die ersten zwei Reihen zuverlässig. Sobald die Matrix um eine Reihe vergrößert wurde, verzerrte sich das Signal, was dazu geführt hat, dass die Reihe zu falschen Taktflanken das falsche Muster darstellte. Die Signalgeschwindigkeit nahm leicht ab, jedoch nicht bemerkbar. Zusätzlich wurde der Stromverbrauch der LED-Matrix gemessen. Es zeigte sich, dass bei einer hohen Helligkeit bereits vier Module einen Strom von knapp 1 A benötigen. Eine hohe Helligkeit

ist unangenehm für die Augen, weshalb die Helligkeit deutlich reduziert wurde, wodurch der Stromverbrauch auf 200 mA gesenkt werden konnte.

Kleine Matrix große Matrix

13.4 Auswertung der Testergebnisse

Die durchgeführten Tests zeigen, dass wesentliche Teile des Systems erfolgreich umgesetzt werden konnten. Insbesondere funktioniert die Personenerfassung sowie die Darstellung der Silhouette auf einer kleinen LED-Matrix zuverlässig. Dadurch konnte die Idee hinter dem Grundkonzept des Black Mirrors nachgewiesen werden. Im Hardware-Bereich wurden alle notwendigen Platinen für die Signalstabilisierung und Signalverbesserung erfolgreich aufgebaut und gelötet. Beim Testen der Matrix mit den Platinen wurden zwei Reihen mit der Verstärkerplatine verbunden und am Anfang vom ersten Modul wurde die Levelshifterplatine ebenso miteingebaut. Es traten jedoch Probleme bei der Signalübertragung auf. Beim Zusammenschalten konnte keine Muster auf die zwei Reihen der LED-Matrix dargestellt werden, wodurch weitere Tests erschwert wurden. Die genaue Ursache konnte im Rahmen der Diplomarbeit nicht analysiert werden, jedoch wird vermutet, dass diese im Bereich der Levelshifterplatine liegt. Aufgrund der begrenzten Zeit bis zur Abgabe der Arbeit konnte dieses Problem nicht behoben werden. Dennoch zeigen die bisherigen Ergebnisse und Simulationen, dass der Ursache der Fehler bei den Platinen stattfindet. Eine vollständige Umsetzung des Black Mirrors würde eine weiterführende Analyse und Optimierung der Platinen erfordern. Insgesamt konnte eine kleine Version des Black Mirrors realisiert werden, welche die grundlegende Funktion des Systems demonstriert, jedoch weiteres Erweiterungspotenzial zeigt.

14 Wirtschaftliche Betrachtung

14.1 Kostenanalyse

	BESCHREIBUNG	MENGE	TEILENUMMER	HERSTELLER- TEILENUMMER	Website	STÜCKPREIS	Versand	GESAMT
1	Raspberry Pi 5 8 GB	1	Raspberry Pi 5	Raspberry Pi	Amazon	€ 90,45		€ 90,45
2	Twozoh Micro HDMI auf HDMI Kabel 1M	1	1M	Twozoh	Amazon	€ 9,89		€ 9,89
3	Raspberry Pi 5 USB-C Netzteil	1	EB82182	Raspberry Pi	Amazon	€ 15,90		€ 15,90
4	Miuzei Gehäuse für Raspberry Pi 5	1	Boîtier pour Raspberry Pi 5	Miuzei	Amazon	€ 10,99		€ 10,99
5	ZDE 2 Stück Kamera Kabel CSI FFC	1	ZDE 2PCS	ZDE	Amazon	€ 7,99		€ 7,99
6	Raspberry Pi Camera Modul 3 Wide NoIR	1	SC0875	Raspberry Pi	Amazon	€ 30,99		€ 30,99
7	1 Stück MAX7219 Dot Matrix Modul	40	----- -----	-----	AliExpress	€ 2,65	€ 47,24	€ 152,63
8	10 Stück 2,54mm/2mm u-Form einreihiger Stecker	1	RA DIP 2.54mm Pin Header	Yu Na Sheng	AliExpress	€ 4,89	€ 4	€ 8,89
9	Arceli 120 Stück 2,54 mm Gerade einreihige Stieftleisten	1	00 454	Arceli	Amazon	€ 9,99		€ 9,99
10	IC OPAMP VFB 1 CIRCUIT SOT23-5	20	296-13033-1-ND	OPA354AIDBVT	DigiKey	€ 1,769		€ 35,38
11	RES 10K OHM 5% 1/4W AXIAL	40	CF14JT10K0CT-ND	CF14JT10K0	DigiKey	€ 0,0124		€ 0,50
12	RES 47 OHM 10% 2W AXIAL	20	BC3362CT-ND	PR02FS020470 9KR500	DigiKey	€ 0,239		€ 4,78
13	IC BUFF NON-INVERT 5.5V 14-PDIP	3	296-4655-5-ND	SN74AHCT125N	DigiKey	€ 0,69		€ 2,07
14	SWITCH ROCKER SPST 10A 125V	1	EG5619-ND	RA1113112R	DigiKey	€ 0,55		€ 0,55
15	+5V2.4A;18AWG/150 CM	1	1866-NGE12E05-P1J-ND	NGE12E05-P1J	DigiKey	€ 9,08		€ 9,08

Die Bestellung bei DigiKey hat zusätzlich noch eine Mehrwertsteuer von 10,47 Euro gekostet. Das Projekt kostet ins gesamt 400,55 Euro. Diese Kosten in dieser Höhe waren zu erwarten.

DigiKey MwSt.	€ 10,47
Gesamtpreis	€ 400,55

!!!!!! Ist noch nicht fertig

15 Zusammenfassung und Ausblick

15.1 Zusammenfassung der Ergebnisse

15.2 Erfüllung der Zielsetzung

15.3 Mögliche Erweiterungen

15.4 Persönliche Reflexion

16 Literaturverzeichnis

17 Abbildungsverzeichnis

<i>Abbildung 1: Abstrahierte Darstellung</i>	7
<i>Abbildung 2: Erode & Dilate Bildliche Darstellung</i>	10
<i>Abbildung 3: Ausgabe des ASCII-Mirrors</i>	14
<i>Abbildung 4: Raspberry Pi 5 Pinout</i>	15
<i>Abbildung 5: Schmid Trigger Schaltung</i>	16
<i>Abbildung 6: Simulation der Schaltung</i>	16
<i>Abbildung 7: OpenCv und Python Logo</i>	18
<i>Abbildung 8: NumPy Logo</i>	18
<i>Abbildung 9: Mediapipe Logo</i>	19
<i>Abbildung 10 : GitHub Logo</i>	20
<i>Abbildung 11: PyCharm Logo</i>	20
<i>Abbildung 12: Ablaufdiagramm des Systems</i>	21
<i>Abbildung 13: Pi Camera Modul 3 Wide NoIR</i>	21
<i>Abbildung 14: Raspberry Pi 5</i>	22
<i>Abbildung 15: MAX7219</i>	22
<i>Abbildung 16: Kamera Kabel</i>	23
<i>Abbildung 17: Netzteil für Raspberry Pi</i>	23
<i>Abbildung 18: HDMI zu HDMI-Mini Kabel</i>	23
<i>Abbildung 19: Case für Raspberry Pi</i>	23
<i>Abbildung 20: Ablaufdiagramm Personenerfassung</i>	24
<i>Abbildung 21: Ablaufdiagramm LED-Ansteuerung</i>	24
<i>Abbildung 22: Bibliotheken Import</i>	34

<i>Abbildung 23: Initialisierung</i>	<i>34</i>
<i>Abbildung 24: drawImage Funktion</i>	<i>35</i>
<i>Abbildung 25: Layout Verbindungsplatine</i>	<i>37</i>
<i>Abbildung 26: 3D Darstellung Verbindungplatine</i>	<i>37</i>
<i>Abbildung 27: OPA354 Schaltzeichen</i>	<i>38</i>
<i>Abbildung 28: OPV-Verstärkerschaltung</i>	<i>39</i>
<i>Abbildung 29: OPV-Verstärkerschaltung Simulation</i>	<i>40</i>
<i>Abbildung 30: Layout der Verstärkerplatine</i>	<i>41</i>
<i>Abbildung 31: 3D Darstellung der Verstärkerplatine</i>	<i>41</i>
<i>Abbildung 32: Schaltzeichen des Levelshifters</i>	<i>42</i>
<i>Abbildung 33: Levelshifter-Schaltbild</i>	<i>42</i>
<i>Abbildung 34: Simulation der Levelshifterschaltung</i>	<i>43</i>
<i>Abbildung 35: Layout der Levelshifterplatine</i>	<i>44</i>
<i>Abbildung 36: 3D Darstellung der Levelshifterplatine.....</i>	<i>44</i>

18 Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne unerlaubte Hilfe angefertigt habe. Nach bestem Wissen und Gewissen habe ich keine anderen als die angegebenen Quellen verwendet. Alle Stellen, die aus veröffentlichten oder unveröffentlichten Quellen entnommen wurden, sind als solche kenntlich gemacht und entsprechend gekennzeichnet.

Ich erkläre außerdem, dass die zur Beurteilung eingereichte Arbeit meine eigene Leistung ist und keine unzulässige Zusammenarbeit enthält, abgesehen von der im Rahmen der Diplomarbeit vorgesehenen Teamarbeit.

Zur Unterstützung bei der Programmierung, der Strukturierung sowie der sprachlichen Ausarbeitung der Diplomarbeit wurden die Tools ChatGPT, Claude und Perplexity verwendet. Diese dienten ausschließlich als Hilfsmittel. Für den gesamten Inhalt der Arbeit übernehme ich die volle Verantwortung.

Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt und ist bisher nicht veröffentlicht worden.

Graz, am

Graz, am